

Speaker Notes - Solution Architecture

Presenter notes extracted verbatim from the three slide sets, listed by set and slide number.

1. Solution Architecture Fundamentals

Slide 1

Welcome to Solution Architecture Fundamentals. We will connect business needs to system boundaries, design decisions and evidence. Our digital loan example and its performance targets are entirely hypothetical teaching material.

Slide 2

By the end of this chapter, participants should be able to connect a business need to a clear solution boundary, explain which requirements influence architecture, and challenge a proposed design using evidence. Reading diagrams is only part of the work. We also need to recognise unanswered questions, distinguish facts from assumptions, and preserve the reasoning behind important decisions. The hypothetical loan application will provide a shared example throughout. The closing exercise asks participants to justify a choice, identify its consequences, and explain how they would validate it.

Slide 3

In our hypothetical journey, the customer signs in, submits an application and follows its progress. Several systems contribute: Bank Identity supports identity, existing customer records supply relevant information, and the Credit Decision Service owns assessment and any external bureau checks. The new Loan Application System coordinates its own responsibilities within that wider environment. The customer experiences one journey, while different teams operate its parts. Architecture makes these dependencies visible so that participants can discuss ownership, failure and communication across the whole journey.

Sources: <https://c4model.com/introduction>

This is a business journey. Tracking can start after receipt while credit assessment continues, so the drawing does not impose a blocking software call sequence.

Slide 4

All details of this case are invented for teaching. We are introducing loan application capture and tracking for existing customers in one pilot country. The new system records applications and exposes their status. Existing Bank Identity, Customer Records / Core and Credit Decision Service remain dependencies. Credit policy and approval stay with the Credit Decision Service, which also handles external bureau checks. We are not replacing core banking or designing collections. This limited scope lets us practise architectural reasoning without claiming to describe the bank's real processes, interfaces or approval policies.

The pilot excludes new-customer onboarding, expansion to other countries, credit-policy changes, core replacement, contract signing, disbursement and servicing. Any customer-benefit measures are hypotheses to validate.

Slide 5

Architecture concerns the organisation of a solution and the significant relationships and choices that shape it. Its description is the set of representations used to explain selected aspects to stakeholders. ISO 42010 explicitly distinguishes these concepts. In our hypothetical case, a context diagram describes the Loan Application System's relationships; it does not itself perform application processing. A decision to retain credit assessment in an existing service changes our proposed organisation of responsibilities. Updating the diagram records that choice. The useful question is whether the description helps people understand, evaluate and maintain the solution.

Sources: <https://www.iso.org/standard/74393.html>

Slide 6

Architecture and detailed design form a continuum rather than two completely separate activities. Consider the reach of a decision: does it influence responsibilities, interfaces, quality requirements or several teams? In our hypothetical pilot, deciding which system owns an application record has broad consequences. Choosing the spacing between two form fields is usually local. Context can change that judgement: even a field name becomes significant when external systems depend on its published meaning. Focus on impact and the cost of change, rather than deciding that architecture simply means large diagrams or senior job titles.

Sources:

<https://www.cognitect.com/blog/2011/11/15/documenting-architecture-decisions>

Slide 7

These three scopes are useful working distinctions, rather than a universal organisation chart. Enterprise architecture considers direction and coherence across the organisation. Solution architecture concentrates on a particular business outcome across the systems and responsibilities needed to deliver it. Software architecture examines the structure within software systems. In our invented example, a bank-wide principle about reusing shared services belongs at enterprise scope; combining existing services with the Loan Application System is solution scope; organising that system's internal responsibilities is software scope. The scopes overlap, and people may contribute across more than one.

Slide 8

A stakeholder concern is something that matters to that person or group; it need not be a complaint. In our hypothetical case, customers want understandable progress, operations colleagues want recoverable processing, and product owners want a useful application journey. The Credit Decision Service owner needs the solution to respect the agreed assessment interface and responsibilities. Technical teams need maintainable implementation choices. These concerns may reinforce or compete with one another. Ask the group: whose perspective is missing from this initial map? Capture missing stakeholders before treating the current view as complete.

Slide 9

Business and technical analysts bring complementary questions to the same proposed solution. In our hypothetical pilot, business analysis clarifies what an application receipt means, which statuses users understand, and which outcomes matter. Technical analysis explores interface behaviour, data ownership, operational dependencies and ways to test the proposal. These are overlapping contributions, not exclusive territories;

actual responsibilities vary between organisations. Their shared task is to make choices understandable and traceable. Before a decision is treated as agreed, establish who owns it, whose input is needed, and what evidence will support it.

Slide 10

Some requirements influence the solution's structure more strongly than others. We use architectural drivers to focus attention on the business outcomes, important functions, quality requirements and constraints that materially affect our choices. In this invented pilot, timely acknowledgement, a durable application record and reuse of the existing Credit Decision Service are important candidates. Do not assume that every requirement has the same architectural weight. Ask which changes would force us to reconsider boundaries, responsibilities or dependencies. Prioritising these drivers gives later option comparisons a reasoned basis.

Sources: https://sei.cmu.edu/documents/5465/2013_018_101_60984.pdf

Slide 11

Functional requirements describe capabilities or behaviour: for example, our hypothetical system must record an application and expose its status. Quality requirements describe properties of that behaviour under stated conditions, such as a measurable response time at a defined workload. The distinction helps us ask complete questions, but some concerns produce both kinds of requirement. Security, for example, may require explicit authentication behaviour and measurable protection or recovery expectations. Avoid treating quality requirements as optional extras. When terminology becomes debatable, return to the practical objective: a clear statement that can guide design and be evaluated.

Sources: https://sei.cmu.edu/documents/5465/2013_018_101_60984.pdf

Slide 12

A quality attribute scenario connects six parts: the source, stimulus, environment, affected artifact, response and response measure. SEI uses this structure to refine quality requirements. Ask what causes the event, what happens, under which conditions, which part is affected, how it responds and how that response will be assessed. The parts belong together: a response-time target without a workload or measurement boundary leaves important questions unanswered. We will apply the structure to our invented loan pilot next. Keep the wording understandable to both business and technical participants.

Sources: https://sei.cmu.edu/documents/5465/2013_018_101_60984.pdf

Slide 13

This is an invented teaching target, not a bank commitment. With healthy intake and storage, at 20 valid submissions per second, the 95th percentile of API acknowledgement time must be no more than two seconds. Measure from arrival of the valid request at the API boundary to return of its reference at that boundary; customer network delay is outside this measure. Create the durable application record before acknowledging receipt. The target does not mean every request finishes within two seconds. It measures acknowledgement, not credit approval. A delayed assessment is shown as Pending assessment.

For the load test, agree duration, payload mix and the treatment of failed or timed-out requests. Timeouts and errors are not successful acknowledgements. This target applies at the API boundary and is separate from any target for completing assessment.

Slide 14

Keep these categories distinct when discussing the invented pilot. Reusing the existing Credit Decision Service is a stated constraint for the exercise. The expected pilot workload is an assumption that needs evidence. Acknowledging a durable application record before assessment completes is a proposed decision with consequences. A constraint limits the options currently available; an assumption remains provisional; a decision records a choice. Give each important item an owner and a reason to revisit it. In a real engagement, confirm the source and authority of constraints rather than presenting convenient design preferences as bank policy.

Slide 15

For our hypothetical case, the solution boundary contains application capture, the application record, coordination of assessment requests and customer-visible status. Bank Identity, Customer Records / Core and the Credit Decision Service remain outside that boundary. Outside means a separate responsibility in this view; it does not automatically mean external to the bank or a different security trust zone. The Credit Decision Service owns any external bureau checks, so we do not draw a new direct bureau dependency. Ask participants to identify one responsibility that would become ambiguous if this boundary were omitted.

Sources: <https://c4model.com/diagrams/system-context>

Slide 16

Use this context view to agree the boundary of the hypothetical pilot: existing retail customers in one country. The Loan Application System captures, saves, submits and tracks applications. Bank Identity and Customer Records / Core remain existing dependencies, while the Credit Decision Service retains authority over credit policy and approval. Any credit-bureau connection belongs behind that service; it is not a new direct integration here. Ask participants to read each relationship aloud and identify its purpose. This view identifies the surrounding people and systems; the submission sequence comes later.

Reference: <https://c4model.com/diagrams/system-context>

The Loan Application System uses Bank Identity for sign-in and obtains authorised customer information from Customer Records / Core. Credit Decision Service owns credit policy and external bureau checks. Relationship arrows denote uses or information exchanges, rather than a detailed request/response sequence. Source: <https://c4model.com/diagrams/system-context>

Slide 17

Ask which question the audience is trying to answer before selecting a view. In this case, a context view clarifies boundaries and dependencies; an interaction view explains submission and assessment; a data view identifies ownership; and a deployment view

examines placement and failure boundaries. These are different representations of the same proposed solution. Keep system names and responsibilities consistent across them. Invite a business analyst and a technical analyst to choose a useful view for the question, 'What happens when assessment is delayed?' Several views may be needed to answer it completely.

Reference: <https://c4model.com/diagrams>

Slide 18

A viewpoint defines conventions for addressing stakeholder concerns; a view applies those conventions to a particular solution. For this course, a context viewpoint asks us to show the system boundary, relevant people, neighbouring systems and labelled relationships. The actual loan-application context diagram is a view produced using those conventions. A viewpoint can guide other projects; this view contains this case's specific elements. Ask participants whether a diagram style alone is sufficient: we also need to know the intended questions, audience and meaning of its elements. Avoid treating 'view' and 'viewpoint' as interchangeable labels.

Background: <https://www.iso.org/standard/74393.html>

Working vocabulary follows the architecture-description distinction in ISO/IEC/IEEE 42010. Viewpoints define the conventions and concerns for descriptions. No particular number of views or mandatory notation is implied. Source: <https://www.iso.org/standard/74393.html>

Slide 19

Choose the amount of detail needed for the decision being discussed. C4 distinguishes system context, containers, components and code. In C4, a container represents an application or data store, not necessarily a Docker container. For our pilot, first agree external responsibilities, then examine the application's major runtime parts, and inspect internal components only where a question warrants it. Separating intake from assessment processing is a responsibility decision; it does not automatically require two microservices. Keep technologies out of a high-level discussion when they do not help the participants assess scope, behaviour or feasibility.

References: <https://c4model.com/diagrams> ; <https://c4model.com/abstractions/container>

A C4 container is an abstraction for an application or data store and does not imply Docker or any particular deployment technology. Source: <https://c4model.com/abstractions/container>

Slide 20

Walk through receipt and assessment as separate milestones. The customer submits a valid application; the Loan Application System durably records it before returning an application reference and 'Pending assessment'. Processing then obtains an outcome from the existing Credit Decision Service and updates the displayed status. The reference confirms receipt, not loan approval. During the exercise's five-minute credit-service outage, healthy intake and storage can retain applications awaiting assessment. Explain that accepted work must remain recoverable after interruption. The required acknowledgement target is p95 API time ≤ 2 s at 20 valid submissions/s in the

agreed load test.

This diagram shows the customer-facing ordering of milestones. It omits the internal dispatch mechanism and correlation details. Recording precedes receipt, and assessment proceeds as separate work. Its outcome can arrive quickly or later. Implementation patterns for reliable handoff belong in the integration chapter. Source: <https://learn.microsoft.com/en-us/azure/architecture/patterns/asynchronous-request-reply>

Slide 21

Clarify what each capability owns before discussing data exchange. The new system owns its application record, reference and workflow status. Existing customer records remain authoritative for customer information, and the Credit Decision Service remains authoritative for assessment outcomes. Define the interface contract for identifiers, required data, validation, status meanings and uncertain responses. Repeating the same customer, key and payload returns the same application reference under the agreed retention period and contract. Reusing a key with changed content needs explicit handling. This is a bounded retry contract, not a promise that every underlying operation executes exactly once.

Reference:

<https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/>

Specify the request-key retention window. Reuse of a key with a different payload needs an explicit error response. Duplicate detection must use an agreed customer and request identity. The new system keeps the application processing status, while Credit Decision Service remains authoritative for the credit decision. Do not promise exactly-once execution. Source:

<https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/>

Slide 22

Compare options against the agreed requirements and constraints before assigning a preference. For this pilot, both options must preserve existing credit-decision ownership and record valid applications correctly. Ask how each behaves when the Credit Decision Service is slow, unavailable or returns an uncertain response. Compare customer clarity, dependency exposure, recovery work and operational effort. Avoid presenting invented costs or delivery estimates as established facts. Mark unknowns and identify the experiment or stakeholder answer needed. The preferred option is the one we can justify for this context; another set of priorities could lead to another defensible choice.

Slide 23

Option A waits for assessment before responding; its submission interaction therefore depends on the Credit Decision Service completing. Option B records the application and acknowledges receipt while assessment proceeds separately. For this case, prefer B because the target concerns durable acknowledgement, not a credit decision within two seconds. The customer sees 'Pending assessment' and can retrieve the application using its reference. Discuss the cost: status handling, recovery and monitoring become explicit responsibilities. The pattern does not accelerate credit assessment, and selecting it does not prove the latency target. Keep the existing service responsible for approval.

Reference: <https://learn.microsoft.com/en-us/azure/architecture/patterns/asynchronous-r>

Both options require safe retry handling and operational visibility. Option B adds explicit coordination of pending work and its customer-visible status.

Slide 24

An architecture decision record captures a significant choice, its context, status and consequences. Read this case's record as a proposal: acknowledge a durably recorded application before assessment completes. Its rationale is the acknowledgement target and the possibility of credit-service delay. Its costs include managing pending work, status visibility and recovery. Ask what evidence is still missing and who must agree the decision within the project. 'Proposed' describes the architecture decision's status, not the customer's application status. If the decision later changes, preserve its reasoning and link to the replacement record.

Reference:

<https://www.cognitect.com/blog/2011/11/15/documenting-architecture-decisions>

The decision owner, reviewers and review trigger must be named in the real engagement. Proposed means the option still needs agreement and evidence. Candidate trigger: failure to meet the acknowledgement target or a changed dependency contract. When a later choice replaces this ADR, retain its history and mark it superseded.

Slide 25

Turn each risk into a concrete validation question. If the receipt response is lost, does a repeated valid submission return the original reference? If credit assessment is unavailable for five minutes, are accepted applications still retrievable as 'Pending assessment'? If an application process restarts, is accepted work recovered? During the agreed load test, measure acknowledgement latency and success at 20 valid submissions/s; fast errors must not count as successful acknowledgements. Also test that one customer cannot read another customer's application. These are scoped checks for the hypothetical solution, not proof of reliability under every possible failure.

Recovery claims apply to the specified failure model. The exercise covers a temporary service outage and a process restart with healthy durable storage. It does not claim survival of every possible storage, regional or infrastructure failure.

Slide 26

Architecture work continues as the solution becomes better understood. During discovery, agree the boundary, concerns and quality scenarios. During design, compare feasible options and record decisions. During delivery, test interfaces and failure behaviour against those decisions. During operation, compare observed behaviour with the agreed expectations and investigate differences. For this case, an increasing number of pending applications might prompt a review of capacity, recovery or ownership. A new business need might change the original trade-off. Keep the relevant views and decision records aligned with changes; an early diagram is not a permanent description of the running system.

Background on iterative design and learning from operation: <https://docs.aws.amazon.com/wellarchitected/latest/framework/general-design-principles.html>

Slide 27

Follow one chain from intention to evidence. The business need is that customers know their application has been received. The requirement specifies a durable record and p95 API acknowledgement ≤ 2 s at 20 valid submissions/s. The proposed decision separates acknowledgement from assessment. Evidence includes the agreed load test and reconciliation of returned references to stored applications. Assign identifiers to the linked items so a change can be followed in either direction. A delayed credit outcome does not automatically violate an acknowledgement requirement. Traceability helps us find the relevant evidence; the links themselves do not establish that the requirement has been met.

Slide 28

Run this exercise for exactly 12 minutes: two minutes to explain the situation, six minutes in pairs, two minutes for reporting and two minutes for the worked debrief. The Credit Decision Service is unavailable for five minutes, intake and storage are healthy, and a customer retries after losing the response. Pairs compare waiting for a decision with recording and acknowledging first. Ask for a choice, the exact customer message, duplicate-handling behaviour and one validation check. Keep the target visible: p95 API acknowledgement ≤ 2 s at 20 valid submissions/s, with durable recording before acknowledgement.

The five-minute outage is a hypothetical scenario, not the duration of the exercise.

Slide 29

A defensible proposal is to record and acknowledge first: 'Application received. Reference L-1042. Status: Pending assessment.' Receipt is not approval. Under the agreed retention period and contract, repeating the same customer, key and payload returns that original reference. The five-minute credit-service outage leaves the application pending; it must not create an approval or decline. Assessment remains with the existing Credit Decision Service when processing resumes. Ask participants what still needs validation: latency at the agreed load, recovery of accepted work and correlation of uncertain outcomes. The answer is a justified proposal supported by planned evidence, not a demonstrated guarantee.

Slide 30

Use these sources as starting points for deeper study, while keeping the case separate from the references. C4 supports discussion of architectural representations; Nygard introduces lightweight decision records; Microsoft illustrates separating acknowledgement from longer processing; and the AWS article examines safe retries. The bank, workflow details, test targets and outage scenario in this chapter are hypothetical. Ask participants to select one artifact they could improve in their own work: a boundary diagram, quality scenario, interface contract or decision record. Their next step should answer a real stakeholder question and identify the evidence needed.

References: <https://c4model.com/diagrams> ;
<https://www.cognitect.com/blog/2011/11/15/documenting-architecture-decisions> ; <https://learn.microsoft.com/en-us/azure/architecture/patterns/asynchronous-request-reply> ;
<https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/>

2. Architectural Design Patterns and Best Practices

Slide 1

This chapter explores architectural design patterns and best practices. We will compare the decisions they address, apply them to our hypothetical loan pilot, and justify choices using requirements and evidence.

Slide 2

The aim is to use pattern names as tools for reasoning. Participants should recognise the problem a pattern addresses, distinguish its essential idea from optional implementation choices, and explain the consequences for customers and delivery teams. We will repeatedly return to the invented loan pilot so the discussion remains concrete. By the end, participants should be able to compare an internal module boundary with a deployment boundary, explain reliable handoff, and identify evidence needed before accepting a proposal. Selecting several fashionable patterns is not the objective; producing a defensible design is.

Slide 3

These concepts answer different questions and can work together. Layers organise technical responsibilities; a modular monolith organises internal capabilities within a common deployment; ports and adapters isolate application logic from external technology. Messaging concerns communication, while CQRS concerns read and write models. In our hypothetical pilot, one application could combine several of these choices without becoming contradictory. Ask participants which decision each label actually describes. This prevents a false comparison in which a layered application, an asynchronous interaction and a microservice are treated as three interchangeable solutions to the same problem.

Slide 4

Read a pattern through its problem, context, proposed structure and consequences. The useful question is not whether its diagram looks familiar, but whether the forces behind it resemble our situation. In the invented pilot, acknowledging receipt before assessment finishes addresses a customer interaction problem. It also creates responsibilities for pending work, truthful status and recovery. A pattern description helps us anticipate these costs; it does not supply evidence that our implementation meets its target. When discussing any candidate, ask what would make it unsuitable and which assumption needs to be tested.

Slide 5

All case details are invented. Existing customers in one country use a new Loan Application System, delivered by one pilot team. It owns receipt, application workflow and status; the existing Credit Decision Service owns assessment and approval. Bank Identity and Customer Records / Core remain dependencies. The illustrative receipt target is p95 API acknowledgement within two seconds at 20 valid submissions per second, with healthy intake and storage. Record the application durably before returning its reference. Pending assessment is an honest processing state, never a credit approval. Independent internal service releases are not yet a demonstrated need.

The receipt target is measured at the application API boundary. Agree test duration, payload mix and the treatment of errors and timeouts; they must not count as successful

receipts. The brief does not define a deadline for completing assessment.

Slide 6

Clear boundaries start with a responsibility that people can explain. Keep closely related behaviour together and make collaboration with other responsibilities explicit. In our hypothetical pilot, application capture should not silently modify credit policy, and a communication component should not invent an assessment outcome. Give each boundary a clear contract, hide implementation details that callers do not need, and identify the information it owns. These principles apply inside one application and between deployed services. Ask what a caller must know to use a responsibility safely, and what knowledge would create unnecessary coupling.

Slide 7

A layer separates a technical responsibility, such as presentation, application processing or data access. A tier describes physical deployment separation; several layers can run together. In our hypothetical pilot, drawing three layers does not commit us to three servers or three services. Explain the allowed dependencies, rather than relying on vertical position alone. Closed layering permits calls only to the next lower layer; open layering permits selected lower layers to be reached directly. Choose and document the convention. Additional logical layers do not automatically add network hops, although separate physical tiers can.

Sources:

<https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/n-tier>

Slide 8

The illustrative modules are Applications, Assessment coordination and Customer communication, inside one deployment. They are a teaching partition, not a universal design. Applications owns application records; assessment coordination interacts with the existing decision authority; customer communication presents appropriate messages. Their contracts should prevent arbitrary access to one another's internals. A single deployment can still run in multiple replicas. For our invented one-team pilot, a modular monolith is a candidate because independent internal releases are not yet justified. Its suitability still depends on workload, maintainability and recovery evidence, rather than on the label alone.

Sources: <https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures>

Slide 9

A port is an application-facing contract; an adapter connects that contract to a particular technology. In this diagram, arrows show code dependencies toward application-owned ports. They do not show the direction of every runtime call: the application can invoke an external adapter through a port. In our hypothetical pilot, an assessment adapter calls the existing Credit Decision Service while application workflow remains independent of that service's transport details. Test adapters help exercise application behaviour in isolation. They do not remove production storage requirements or transfer credit approval authority into the new application.

Sources: <https://alistair.cockburn.us/hexagonal-architecture>

Slide 10

A bounded context identifies where a particular model and its terminology have a consistent meaning. It is not automatically a microservice, database or module. In the invented pilot, the application workflow model includes receipt and Pending assessment; the existing assessment context determines the credit outcome. Translate meanings explicitly when these responsibilities exchange information. The new system may store a received outcome for display without becoming its policy authority. Ask participants whether two teams using the word status necessarily mean the same thing. Understanding that difference can be more important than selecting a transport technology.

Sources: https://www.domainlanguage.com/wp-content/uploads/2016/05/DDD_Reference_2015-03.pdf

Slide 11

Microservices seek useful autonomy through independently deployable services with explicit responsibilities and contracts. Distribution also introduces remote failures and additional operational work. In our hypothetical pilot, the existing bank services remain dependencies regardless of how we organise the new application's internals. We have not yet established a need to release its internal capabilities independently. Avoid promising resilience or faster delivery merely because boxes become separate processes. Ask what autonomy a proposed service actually gains, which coordination remains, and who will operate it. A service boundary is valuable when those answers justify its costs.

Sources: <https://martinfowler.com/articles/microservice-trade-offs.html>

Slide 12

Use the invented one-team pilot as the decision context. Compare release needs, workload differences, ownership, change impact and operational capability. A modular application is a reasonable starting candidate while independent internal deployment has no demonstrated benefit. This does not make it a permanent rule. Record triggers that would reopen the choice, such as measured capacity pressure in one capability or a substantiated need for separate release cadences. Distinguish deployment units from runtime replicas and repository organisation. Choose the structure that meets current requirements while making significant future changes understandable and assessable.

Sources: <https://martinfowler.com/bliki/MonolithFirst.html>

Slide 13

The customer interaction and the background assessment can use different communication arrangements. In our invented pilot, a request receives a brief reply confirming durable application receipt; assessment continues separately through an agreed handoff. Request-reply describes a conversation and is not inherently limited to blocking communication. An asynchronous message does not itself prove successful processing. We still need ownership of pending work, correlation, recovery and a visible outcome. The receipt target applies to the API exchange, not the assessment duration. Keep Pending assessment meaningful when the existing Credit Decision Service is slow or unavailable.

Sources: <https://learn.microsoft.com/en-us/azure/architecture/patterns/asynchronous-request-reply>

Slide 14

A work queue allocates each delivery to one of its competing workers. Publish-subscribe gives each matching independent subscription its own message to process; workers can also compete within a subscription. In the hypothetical pilot, an assessment work item needs one responsible processing path, while a permitted workflow event might support separate communication and reporting subscriptions. Neither arrangement guarantees exactly-once processing. Redelivery can occur in both, so consumers need an explicit duplicate-handling contract. Decide whether we are assigning work or announcing a fact before drawing the messaging topology.

Sources:

<https://learn.microsoft.com/en-us/azure/architecture/patterns/competing-consumers> ;
<https://learn.microsoft.com/en-us/azure/architecture/patterns/publisher-subscriber>

Slide 15

CQRS separates the models used to change information from those used to read it. In our hypothetical pilot, submission logic validates and records an application, while a status model provides a convenient customer view. Both models can share a database and deployment. Separate endpoints alone do not explain the architectural distinction, and event sourcing is optional. If a read projection updates asynchronously, explain its possible delay and avoid contradicting an already issued receipt. The status model reports an assessment outcome received from the existing service; it does not acquire authority to approve credit.

Sources: <https://learn.microsoft.com/en-us/azure/architecture/patterns/cqrs>

Slide 16

Event sourcing makes recorded domain events the basis for reconstructing state. Ask whether the pilot needs that capability, or simply needs an understandable history of application changes. These are different requirements. A conventional application record plus a carefully designed history may meet the stated teaching case. Choosing event sourcing would introduce questions about event evolution, replay, snapshots and external effects. Replaying a saved application must not silently request another assessment or resend a customer notification. Credit policy still belongs to the Credit Decision Service. Invite analysts to name the business question that would justify this additional complexity.

Reference: <https://martinfowler.com/eaDev/EventSourcing.html>

The three events illustrate an application still awaiting assessment. The projection derives state from the relevant stream, rather than from one isolated event. This pattern is an option under discussion, not the selected storage model for the pilot. An `AssessmentOutcomeReceived` event would record a result supplied by the authoritative Credit Decision Service.

Slide 17

Expose the vulnerable gap: an application is saved, the process crashes, and its assessment message is never published. The proposed outbox places the application record and outgoing message in one local database transaction. Successful acknowledgement follows that durable commit. A relay later publishes committed messages and records its progress. The relay can publish a duplicate after a crash, so downstream handling needs its own duplicate protection. Ask learners to draw the transaction boundary around storage only; it does not include the broker or Credit Decision Service. Assign an owner to monitor message age and investigate stalled handoffs.

Reference: <https://microservices.io/patterns/data/transactional-outbox.html>

Slide 18

A retry needs an explicit identity contract. For this pilot, the same customer, key and payload returns the original application reference within the agreed retention period; a reused key with changed content has a defined error response. Protect concurrent attempts atomically. On message consumption, store the duplicate marker and local business update in the same transaction. A separate check followed by an update leaves a race. This local protection does not make an external Credit Decision Service call safe to repeat: agree its request identity and uncertain-outcome handling separately.

References:

<https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/> ; <https://microservices.io/post/microservices/patterns/2020/10/16/idempotent-consumer.html>

Slide 19

Use a separate hypothetical appointment journey to explain a saga. Reserving a slot commits one local transaction; assigning an adviser commits another. If adviser assignment cannot complete, releasing the reserved slot is a business compensation. Ask what happens if release also fails, or another customer has already taken a newly available slot. Record progress and define recovery responsibility. A notification already sent cannot be unsent, although a correction may be sent. Compensation therefore requires business meaning, not an assumption of automatic rollback. The loan pilot's asynchronous assessment alone does not establish a need for a saga.

References: <https://microservices.io/patterns/data/saga.html> ;

<https://learn.microsoft.com/en-us/azure/architecture/patterns/compensating-transaction>

Slide 20

An anti-corruption layer protects the application's vocabulary when another system uses different meanings. In our pilot, an adapter translates the existing credit service's identifiers, statuses and errors into the application's agreed model. Have analysts examine ambiguous terms such as accepted: does it mean a request was received, or a loan was approved? An unknown response must have an explicit handling path, never an invented approval. Credit policy remains owned by the Credit Decision Service.

Translation can live inside the application; a separate deployed service is not required. Validate the mapping with examples and contract tests.

Reference:

<https://learn.microsoft.com/en-us/azure/architecture/patterns/anti-corruption-layer>

Slide 21

This is a separate future migration example, not an extra requirement for the pilot. Imagine gradually replacing an old loan-intake application while the existing core and

credit service remain. Route a clearly bounded intake capability to the new implementation, observe it, then expand deliberately. Before moving a route, identify who owns each record and how status remains consistent across old and new paths. Rolling traffic back will not automatically reverse data already written. Ask for migration evidence, reconciliation and a rollback plan that addresses those writes. Define when an old route can safely be retired.

References: <https://martinfowler.com/bliki/StranglerFigApplication.html> ;
<https://learn.microsoft.com/en-us/azure/architecture/patterns/strangler-fig>

Slide 22

Separate three responsibilities: timeouts bound a wait, retries repeat eligible attempts, and a circuit breaker temporarily limits calls to a failing dependency. For the pilot, assessment dispatch needs a total time budget, bounded retries with backoff and jitter, and an agreed recovery path. A timeout leaves the previous outcome uncertain. Retry only where the operation's contract makes repetition safe. Ordinary credit declines are business outcomes, not technical failures that should trip the breaker. While assessment is unavailable, a recorded application remains Pending assessment. Demonstrate recovery probes and check that retries at several layers do not multiply unexpectedly.

References:

<https://learn.microsoft.com/en-us/azure/architecture/patterns/circuit-breaker> ; https://docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/rel_mitigate_interaction_failure_limit_retries.html

Retry policy includes exponential backoff, jitter and a maximum attempt count or total deadline. Avoid independent retries multiplying across layers. A circuit breaker opens after a configured technical-failure condition, waits, then admits a limited set of probes in its half-open state. A successful credit decline is not a failed technical call. These mechanisms cannot make an unsafe external side effect safe to repeat.

Slide 23

A bulkhead gives one workload a resource budget so its exhaustion is less able to consume another workload's capacity. In the pilot, bound assessment workers separately from intake and status handling. Then inspect the resources still shared: database connections, processor capacity and queues can undermine the intended separation. Ask which resource the diagram actually isolates. Reserving capacity has a cost and needs measurement. Test a saturated assessment path while measuring acknowledgement at the application API boundary: $p_{95} \leq 2$ s at 20 valid submissions/s, with intake and storage healthy. This is a hypothetical acceptance target, not a guaranteed result.

Reference: <https://learn.microsoft.com/en-us/azure/architecture/patterns/bulkhead>

Slide 24

Use low-risk product reference information to illustrate cache-aside. The application checks the cache, loads a missing value from its authoritative source and caches that value. Analysts must define how stale the information may be, what happens on source failure and how changes become visible. A typical update flow changes the source before invalidating its cache entry, but concurrent activity still needs consideration. Ask whether an outdated label is acceptable and whether an outdated eligibility statement would mislead a customer. Do not treat a cached value as credit approval. Observe freshness and source load as well as cache hit rate.

Reference: <https://learn.microsoft.com/en-us/azure/architecture/patterns/cache-aside>

Slide 25

For one delivery team serving existing customers in one pilot country, propose a modular monolith with explicit internal responsibilities. Ports and adapters isolate external contracts. An outbox supports durable handoff; idempotency addresses repeated submissions and message delivery; bounded dependency calls protect recovery. These choices form a candidate design to validate, not a promise of performance. Keep approval and credit policy in the existing Credit Decision Service. The brief does not establish a need for microservices, saga coordination or event sourcing. Ask learners to connect each selected pattern to a stated failure or change scenario and identify its operating cost.

Reference: <https://microservices.io/patterns/data/transactional-outbox.html>

Slide 26

Read each anti-pattern through its consequence. Pattern shopping adds mechanisms before a requirement justifies them. A distributed monolith pays for remote calls while preserving coordinated releases. Shared writable data lets another responsibility bypass the rules of its owner. Unlimited retries can amplify an outage. Ask learners to describe one observable customer or operational effect and the smallest corrective change. For the pilot, examine whether a new service can really be released independently and whether an assessment timeout can generate uncontrolled retry traffic. The pattern name is a prompt to investigate, not sufficient evidence of a defect.

References: <https://martinfowler.com/articles/microservice-trade-offs.html> ; https://docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/rel_mitigate_interaction_failure_limit_retries.html

Slide 27

Turn best practices into evidence requests. Link each design choice to a scenario, an accountable owner and an observable result. For the pilot, review transaction boundaries, interface examples, duplicate handling, message age, recovery procedures and the meaning of customer statuses. Exercise a crash between durable commit and publication, concurrent repeated requests, and dependency saturation. Record what was measured and under which conditions; a successful demonstration is evidence for that scenario, not proof of universal reliability. Ask what result would cause the team to revise its decision. Keep assumptions visible and revisit them when scope or load changes.

Reference: https://docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/rel_mitigate_interaction_failure_limit_retries.html

Background on design principles:

<https://learn.microsoft.com/en-us/azure/architecture/guide/design-principles/>

Slide 28

Give groups exactly 12 minutes: two to understand the failure, five to design, three to select evidence and two to report. In this hypothetical pilot, the application is saved, then the process crashes before event publication. After restart, the relay may publish twice. One delivery team serves existing customers in one country. Ask for selected patterns, the precise transaction boundary, a customer message and two tests. Preserve durable receipt before acknowledgement and Pending assessment. The target is p95

acknowledgement ≤ 2 s at 20 valid submissions/s at the application API boundary, with healthy intake and storage.

Reference: <https://microservices.io/patterns/data/transactional-outbox.html>

Slide 29

A defensible answer commits the application and outbox message atomically. The relay may repeat publication; a consumer commits its duplicate marker and local business update together. External assessment calls require their own safe-retry contract.

Acknowledgement confirms recording, with status Pending assessment; the reference is not loan approval. Test one: crash after commit and confirm the saved handoff resumes. Test two: redeliver concurrently and confirm one local business effect. Include response semantics and observable evidence. A modular monolith remains feasible for this brief; microservices are not required. Name recovery ownership and the retention period for repeated customer requests.

References: <https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html> ; <https://microservices.io/post/microservices/patterns/2020/10/16/idempotent-consumer.html>

Also test a relay crash after publishing but before recording publication progress: the event may be sent again. Verify that concurrent redelivery does not duplicate the consumer's local effect. This does not prove that an external service call is safe to repeat. Check that service's identity, retry and uncertain-outcome contract separately.

Slide 30

Use these primary references to check pattern intent, applicability and limitations. Cockburn explains application ports and adapters; Evans defines domain models and bounded contexts. Hohpe and Woolf describe messaging patterns, and Richardson describes service collaboration and reliable handoff. Fowler discusses the costs of service autonomy, while Microsoft provides applied pattern guidance. The pilot, numerical targets and exercise are hypothetical teaching choices, not published bank policy. When writing a decision, connect the relevant source to a local assumption and validation evidence. The source explains the pattern; the team must establish that its implementation meets the agreed need.

References: <https://alistair.cockburn.us/hexagonal-architecture> ; https://www.domainlanguage.com/wp-content/uploads/2016/05/DDD_Reference_2015-03.pdf ; <https://www.enterpriseintegrationpatterns.com/patterns/messaging/toc.html> ; <https://microservices.io/patterns/index.html> ; <https://martinfowler.com/articles/microservice-trade-offs.html> ; <https://learn.microsoft.com/en-us/azure/architecture/patterns/>

3. Cloud Native Application Architecture

Slide 1

This chapter connects cloud platform capabilities with the design and operation of the Loan Application System. We will examine responsibility, durable state and controlled change, using the same hypothetical banking pilot throughout.

Slide 2

Use the four objectives as questions to bring into an architecture discussion. Which business outcome should a platform capability improve? Who remains responsible when a managed service is selected? What information must survive when an instance disappears? What evidence would justify the proposed design? Analysts do not need to administer a cluster to ask these questions well. They do need to distinguish the application's behaviour from the infrastructure that hosts it. Throughout the chapter, relate technical choices to a durable receipt, truthful processing status and controlled interaction with the existing Credit Decision Service. The final exercise combines these concerns under a changing workload.

Slide 3

CNCF describes cloud-native practices across public, private and hybrid environments, with automation and loosely coupled systems supporting repeatable change. Its technology examples are a menu of possibilities, not a mandatory checklist. Kubernetes, containers and microservices can help, but none alone demonstrates that an application meets its business objectives. In our pilot, the modular application can retain one common release while using replaceable instances, external state and managed services. Ask learners which outcome would improve if the application were merely moved unchanged onto a cloud virtual machine. Hosting location may change immediately; predictable recovery and safe change still require deliberate design.

Sources: <https://github.com/cncf/toc/blob/main/DEFINITION.md>

Slide 4

Read these options as operating arrangements, with overlapping capabilities. A managed application platform may use Kubernetes underneath without exposing its administration to the team. Virtual machines offer operating-system control and leave more maintenance work with the customer. Managed Kubernetes exposes orchestration choices while retaining workload and some cluster responsibilities. Functions simplify execution of handlers, but concurrency, duration, startup behaviour and connectivity need service-specific checks. Across all options, the application team still owns its code, data use and access decisions. For the pilot, compare the controls actually needed with the effort needed to operate them. A more configurable platform is useful only when that control serves a requirement.

Sources: <https://learn.microsoft.com/en-us/azure/architecture/guide/technology-choices/compute-decision-tree>
<https://aws.amazon.com/compliance/shared-responsibility-model/>

Slide 5

Every detail of this pilot is invented for teaching. The customer is already known to the bank and submits an application in the single pilot country. The new system owns intake,

the application record, workflow and customer-visible processing status. Credit policy, approval and bureau checks remain with the existing Credit Decision Service. Our illustrative baseline is a p95 acknowledgement time of at most two seconds at twenty valid submissions per second, with healthy intake and durable storage. Assessment can finish later, so the receipt must not imply approval. Ask learners to explain the difference between confirming receipt and promising a credit outcome.

The assumed platform must support replicated instances, bounded background work and the bank's required private connectivity. Data location, retention and access requirements still need the appropriate bank owners' agreement. The target refers to successful acknowledgements measured at the application API boundary. Errors and timeouts must not count as successful receipts.

Slide 6

Use this view to locate the application's dependencies and ask who owns each contract. Customer channels reach the application through an entry point. The application uses existing identity and customer information, stores its own records, and requests assessment from the Credit Decision Service. The arrows identify dependencies; they do not prescribe message order or the physical network layout. For each relationship, ask about access, failure behaviour, capacity and the evidence required before use. A backing service is a network-accessible capability consumed by the application, such as storage or messaging. Moving compute to a managed platform does not automatically move the existing bank systems or their business responsibilities.

Sources: <https://12factor.net/backing-services>

The two boxes are replicas of one versioned application. Each instance can host API handling and bounded background work. They share a common release, rather than acquiring separate microservice ownership. The database transaction records the application and its outbox entry together. Relay and consumer operations require the concurrency and idempotency contracts discussed in Chapter 02. The message channel is a backing service. This dependency view omits the detailed publish/consume sequence.

Slide 7

Distinguish the package from its execution. An image contains the files and dependencies used to start containers; a container runs processes from that image under the runtime's isolation controls. Several instances can therefore use the same image version. An orchestrator observes the running system and acts to approach the desired state. If two instances are requested and one disappears, replacement is attempted, subject to available capacity and platform conditions. This does not restore unsaved application records or prove that both instances can serve customers correctly. Kubernetes provides this kind of orchestration, while managed platforms may expose a simpler interface. Ask what must survive independently when an instance is replaced.

Sources:

<https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-an-image/>
<https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/>
<https://kubernetes.io/docs/concepts/architecture/controller/>

Slide 8

Stateless compute still processes information; it simply does not rely on local memory or ephemeral files as the authoritative record between requests. In this invented example, application L-1042 and its Pending assessment status were committed before instance A stopped. A ready instance can reload that record and continue according to the workflow contract. If the first response was lost, a retry needs a stable request identity so that it does not create another application. Multiple instances also need safe concurrent updates and work claiming. External storage therefore enables replacement, but correct business effects still depend on application rules, transactions and retry handling. A database backup addresses a different failure boundary.

Sources: <https://12factor.net/processes>

Slide 9

Use configuration for values that differ between environments, such as service endpoints and capacity settings. Supply these separately from the application build and validate changes before use. Credentials need stronger controls: restricted access, protected storage, rotation and protection from accidental logging. Where the selected platform and destination support it, workload identity can replace an application-managed password with platform-issued credentials. The identity still needs explicit, limited permission at the destination. In our pilot, permission to read customer details must not imply permission to change credit policy. Ask who can alter configuration, grant access and rotate a remaining secret, and what evidence shows those changes worked without interrupting receipt handling.

Sources: <https://12factor.net/config>

<https://learn.microsoft.com/en-us/entra/identity/managed-identities-azure-resources/overview>

<https://docs.aws.amazon.com/secretsmanager/latest/userguide/best-practices.html>

Slide 10

Build a versioned artifact, evaluate it, then promote the identified artifact through environments while supplying configuration separately. For container images, a digest identifies content more precisely than a tag that can be reassigned. Keep the source revision, test results, security checks and approval evidence linked to that artifact. Provenance records how an artifact was produced; it supports verification without proving that the application is free of defects. For the pilot, reviewers should be able to identify the version handling a particular incident and find a compatible recovery option. Rebuilding independently for production can weaken that connection to the tested artifact. The release record also needs the configuration and data compatibility context.

Sources: <https://12factor.net/build-release-run>

<https://docs.docker.com/dhi/explore/security-concepts/digests/>

<https://slsa.dev/spec/v1.2/provenance>

Slide 11

Use the rows as separate operational decisions. Ask learners to propose checks for initial configuration, local process progress and eligibility to receive application requests. A responding health endpoint is not proof that an individual submission has committed. For the pilot, demonstrate replacement while requests and background jobs are active.

Follow one application reference before and after the change, inspect its durable record and establish which instance may continue its workflow. Include a sudden interruption as well as a planned replacement. The test should show that unfinished work can be recovered under the agreed request identity and duplicate-handling contracts.

Primary sources:

<https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/>

<https://12factor.net/disposability>

Startup or liveness failures act after configured thresholds and follow the container restart policy. Readiness failure does not itself restart the container. In this pilot, credit-service unavailability should not automatically fail intake liveness. Intake may remain ready while it can durably accept work within its admission limits. Draining still needs a bounded termination period and safe handling of work interrupted at the deadline.

Slide 12

The 90/10 traffic split is illustrative. A suitable initial share and observation period depend on the change, exposure and available evidence. Compare candidate and established versions using successful receipts, latency and the age of pending assessments. Define stop criteria and a recovery owner before increasing exposure. Both versions may access the same durable records, so check schema, message and interface compatibility. Returning traffic to the earlier image does not undo writes already committed by the candidate. Data repair or recovery needs a separate, deliberate plan. Ask learners what evidence would justify promotion and what would prevent a safe return to the previous version.

Primary sources:

<https://learn.microsoft.com/en-us/azure/well-architected/operational-excellence/safe-deployments>

Slide 13

Autoscaling changes capacity after signals are measured and new instances become ready. It cannot guarantee an immediate response to a spike. Ask which bottleneck each signal suggests: high CPU, growing queue age and slow receipt responses require different investigations. In this pilot, adding replicas also adds potential background workers. Enforce the credit service's agreed concurrency or rate limit across all instances, rather than multiplying that allowance per replica. Bound database connections, instance count and backlog admission. If safe capacity is exhausted, apply the defined overload response. Test scaling, saturation and subsequent recovery together, including the effect of removing an instance.

Primary sources:

<https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/>

<https://learn.microsoft.com/en-us/azure/architecture/best-practices/auto-scaling>

<https://learn.microsoft.com/en-us/azure/architecture/patterns/throttling>

Slide 14

An SLI measures a defined service outcome; an SLO sets its target and evaluation window. Metrics, logs and traces help explain why outcomes change. Our p95 receipt target of ≤ 2 seconds at 20 valid submissions/s is an illustrative load-test requirement with healthy intake and storage. A production SLO still needs an agreed population and time window. Pair receipt latency with successful acknowledgements and pending-work age: fast errors must not make the service appear healthy. Specify measurement at the application API boundary. Correlate an application across asynchronous handoffs without recording sensitive payloads. Ask which signal would reveal applications that were recorded but never assessed.

Primary sources:

<https://sre.google/sre-book/service-level-objectives/>

<https://opentelemetry.io/docs/concepts/signals/>

Slide 15

Separate continued service from recovery of durable state. Replicas placed in one failure domain may fail together; distributing them can address that exposure. It does not establish that accidentally deleted or corrupted records can be restored. Test instance replacement, relevant domain failure and data restoration as distinct scenarios, with named owners and agreed recovery objectives. Managed services change the responsibility split, which must be checked for the selected offering. For this pilot, review scoped access, approved network paths, data controls and artifact verification with the appropriate bank owners. These are design questions, not a claim that the hypothetical architecture satisfies bank policy or regulatory obligations.

Primary sources:

<https://learn.microsoft.com/en-us/azure/well-architected/reliability/disaster-recovery>

<https://aws.amazon.com/compliance/shared-responsibility-model/>

Slide 16

Compare total operating effort over the expected workload, including a quiet period and a peak. The service bill includes more than active compute: examine idle capacity, storage, network traffic and telemetry, alongside support and recovery work. Container standards help package and run an application consistently. They do not move its database semantics, identity integration or private connectivity automatically. Ask what must change to run this pilot elsewhere and how its records would be moved and verified. A managed capability may reduce some operating work while creating an integration dependency. Record that trade-off explicitly, and test an exit path where the business requires it.

Primary sources:

<https://opencontainers.org/about/overview/>

<https://learn.microsoft.com/en-us/azure/well-architected/cost-optimization/tradeoffs>

Slide 17

Present ADR-003 as a proposal conditional on evidence. One team and one pilot country make a managed application platform a candidate, provided it supports the workload's actual needs. Verify replicated execution, bounded background work, durable backing services, private connectivity, lifecycle controls and usable telemetry. Preserve one common application release; this decision does not require separate microservices.

Credit assessment and approval remain with the existing Credit Decision Service. Ask learners to identify a failed assumption that would trigger reconsideration, such as unsupported worker behaviour or insufficient control over replacement. Record an owner and review trigger instead of treating a platform label as proof of suitability.

Primary sources:

<https://learn.microsoft.com/en-us/azure/architecture/guide/technology-choices/compute-decision-tree>

Slide 18

Allow exactly 12 minutes: two to understand the scenario, five to design, three to specify evidence and two to report. The hypothetical arrival rate rises from 20 to 80 submissions/s for five minutes while one instance is replaced. Credit capacity remains limited. The baseline p95 $\leq$ 2-second receipt target applies only at 20 valid submissions/s with healthy intake/storage. Groups must define and test peak admission behaviour, including response targets and overload handling; they must not assume the baseline extends to 80. Require a durable-state boundary, capacity and lifecycle controls, customer status and a recovery owner. Acknowledge admitted applications only after durable commit; delayed assessment remains Pending assessment.

Primary sources:

<https://learn.microsoft.com/en-us/azure/architecture/patterns/throttling>

<https://12factor.net/disposability>

Slide 19

A defensible answer preserves records and request identity outside replaceable instances. Scale within shared database and credit-service limits, and define what happens when admission capacity is exhausted. Accepted, durably recorded applications retain their reference and show Pending assessment while delayed. If a new request cannot be admitted, return the agreed overload response without claiming successful receipt. Test planned draining and abrupt interruption, including a lost response after commit and a safe retry. Measure baseline performance separately from the proposed peak targets. At 80 submissions/s, report admitted and rejected rates, receipt latency, pending-work age and recovery time. Evidence must establish customer outcomes as well as instance counts.

Primary sources:

<https://learn.microsoft.com/en-us/azure/architecture/patterns/throttling>

<https://12factor.net/disposability>

Slide 20

Use the references to check definitions, service behaviour and the limits of a proposed design. CNCF and the Twelve-Factor App introduce principles. Kubernetes illustrates runtime controls; OpenTelemetry describes telemetry signals. Architecture guidance helps compare choices, while the shared responsibility model highlights work that remains with the customer. Detailed topic links accompany the relevant notes. The platform, traffic peak and performance target in this chapter are hypothetical teaching choices, not evidence about a particular bank or provider contract. Encourage learners to connect each selected capability to an explicit requirement, an accountable owner and a test. Use these sources to deepen the discussion of security, recovery and

infrastructure automation.

Primary sources:

<https://12factor.net/disposability>

<https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/>

<https://opentelemetry.io/docs/concepts/signals/>

<https://learn.microsoft.com/en-us/azure/architecture/guide/technology-choices/compute-decision-tree>

<https://aws.amazon.com/compliance/shared-responsibility-model/>